

Session : Principale
Régime : RM
Enseignant : Sofiane HACHICHA
Classe(s) : CPI2
Barème : 4.5+7.5+6+2

Date : 24/05/2022
Matière : Programmation Java 2
Durée : 90 minutes
Documents : Non autorisés
Nombre des pages : 4

Examen

Exercice 1

Soit les classes suivantes :

- La classe **Personne** qui contient la liste de ses enfants (eux-mêmes de type Personne).
- La classe **TestSerialisation** qui sérialise les objets dans le fichier "C:\poo2\Personnes.bin".
- La classe **TestDeserialisation** qui lit le fichier précédent pour retrouver les objets sauvegardés dans l'état attendu.

On fournit ci-dessous ces différentes classes.

```
import java.io.Serializable;  
import java.util.*;
```

```
public class Personne implements Serializable {  
    private String prenom;  
    private String nom;  
    private List<Personne> enfants = new ArrayList<>();  
  
    public Personne(String prenom, String nom) {  
        this.prenom = prenom;  
        this.nom = nom;  
    }  
    public String getNom() {  
        return nom;  
    }  
    public String getPrenom() {  
        return prenom;  
    }  
    public void ajouterEnfants(Personne... enfants) {  
        // A compléter : Il est à rappeler que l'opérateur à trois points ... (appelé Varargs) permet  
        // d'accepter un nombre variable d'arguments dans une méthode.  
    }  
    public List<Personne> getEnfants() {  
        return enfants;  
    }  
    public String toString() {  
        return this.prenom + " " + this.nom;  
    }  
}
```

```
import java.io.*;
import java.nio.file.*;

public class TestSerialisation {
    public static void main(String[] args) throws IOException {

        Personne p = new Personne("Papa", "Ben");
        p.ajouterEnfants(new Personne("A", "Ben"), new Personne("B", "Ben"), new Personne("C", "Ben"));

        // A compléter : Sérialiser seulement l'objet p (la personne (père) et ses enfants) et
        // accéder au fichier demandé à partir de son Path sans se préoccuper du séparateur « \ »
        // en utilisant la méthode statique get de la classe java.nio.file.Paths.
    }
}
```

```
import java.io.*;
import java.nio.file.*;

public class TestDeserialisation {
    public static void main(String[] args) throws IOException, ClassNotFoundException {

        // A compléter : Accéder et lire le fichier en question à partir de son Path sans se préoccuper
        // du séparateur « \ » et afficher l'objet p sérialisé précédemment (afficher le père puis ses
        // enfants en invoquant la méthode getEnfants() de la classe Personne).
    }
}
```

1. Compléter le code de la classe **Personne**
2. Compléter le code de la classe **TestSerialisation**
3. Compléter le code de la classe **TestDeserialisation**
4. Comment peut-on exclure la liste des enfants de la sérialisation/désérialisation ?

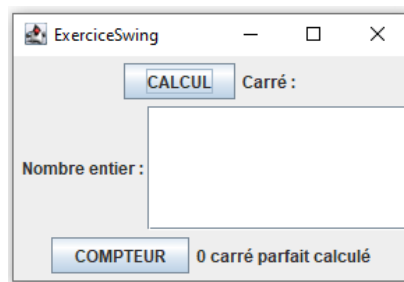
Exercice 2

Écrire un programme qui permet à l'utilisateur de saisir un **nombre entier** dans un champ de texte (JTextField), qui affiche le carré de ce nombre lorsqu'il clique sur le bouton CALCUL et qui incrémente un compteur à chaque fois qu'un carré parfait (le carré d'un entier) est calculé. La valeur de ce compteur est retournée, dans une étiquette, lorsque l'utilisateur clique sur le bouton COMPTEUR.

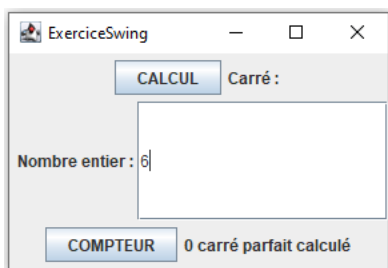
Le programme devra gérer convenablement le cas où l'utilisateur clique sur le bouton CALCUL après avoir saisi une autre chose qu'un nombre entier dans le champ de texte ; le programme remettra, dans ce cas, le champ de texte à blanc et affichera une étiquette vide (à côté du bouton CALCUL) et un message d'erreur sur la console.

Il est important de choisir le (les) gestionnaire(s) d'agencement approprié(s) de manière à avoir les interfaces graphiques comme exposées ci-après.

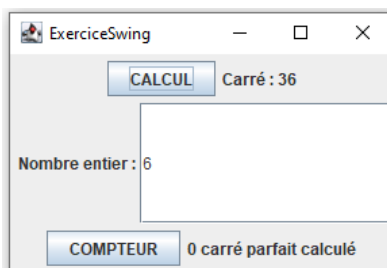
Des scénarios d'exécution de cette application sont présentés dans le tableau 1.



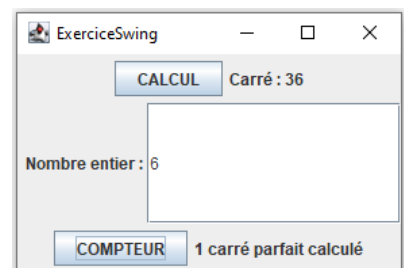
(a) Aperçu de l'interface graphique lors du démarrage de l'application



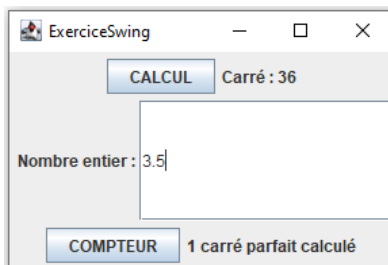
(b0) Aperçu de l'interface après avoir saisi un entier



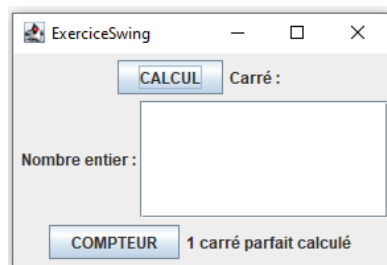
(b1) Aperçu de l'interface après avoir cliqué sur le bouton CALCUL



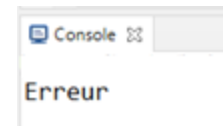
(b2) Aperçu de l'interface après avoir cliqué sur le bouton COMPTEUR



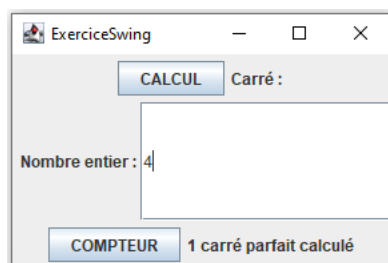
(c0) Aperçu de l'interface après avoir saisi un nombre non entier (3.5)



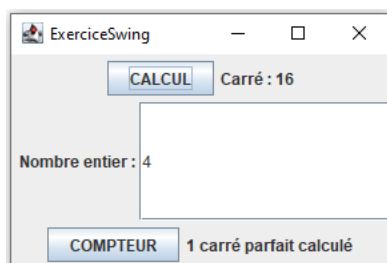
(c1) Aperçu de l'interface après avoir cliqué sur le bouton CALCUL



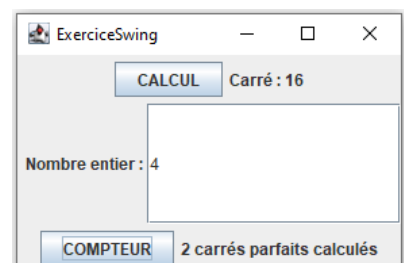
(c2) Aperçu de la console



(d0) Aperçu de l'interface après avoir saisi un deuxième entier



(d1) Aperçu de l'interface après avoir cliqué sur le bouton CALCUL



(d2) Aperçu de l'interface après avoir cliqué sur le bouton COMPTEUR

Tableau 1 – Scénarios d'exécution de l'application ExerciceSwing

Exercice 3

Une agence de voyage désire informatiser la gestion des voyages qu'elle propose. Elle dispose d'une base de données MySQL nommée **testdb** (**user** désigne le nom d'utilisateur de cette BD et **isimg** désigne son mot de passe) et elle veut créer la table suivante :

table_vol (**numVol**, **villeDepart**, **villeDestination**, **dateVol**).

Le champ **numVol** (nombre entier) est défini comme un champ de clé primaire. Les champs **villeDepart** et **villeDestination** permettent de stocker des chaînes de caractères tandis que le champ **dateVol** est de type DATE.

Écrire un programme Java qui permet de :

1. Établir la connexion à la base de données testdb à partir de la chaîne url suivante :
"jdbc:mysql://localhost:3306/testdb"
2. Créer la table **table_vol** présentée ci-dessus.
3. Insérer dans cette table les vols suivants :

numVol	villeDepart	villeDestination	dateVol
100	Paris	Athens	2022-05-19
102	Paris	London	2022-05-17
103	Tunis	Paris	2022-06-25
106	New York	Los Angeles	2022-06-12

4. Affecter une nouvelle villeDestination (Lyon) pour le vol 103.
5. Afficher les dates et les numéros des vols au départ de Paris. L'affichage se fera dans la console.
6. Insérer un nouveau vol (109, Tunis, Alger) sachant qu'aujourd'hui est la dateVol.
La solution proposée doit être basée sur une requête préparée (PreparedStatement).

Exercice 4

Écrire le code de la méthode

```
public static int nbLignesPaires(String fiche)
```

permettant de compter le nombre de lignes paires dans un fichier texte nommé fiche.

Cette méthode ne doit pas lancer des exceptions IOException : s'il y a un problème, il faut retourner la valeur -1.

BON TRAVAIL